

```

1 %=====
2 % Deterministic Ramsey Growth Model
3 %=====
4
5 clear
6 clc
7
8 %Printing the outputs as a text file if ==1
9 output_file=0;
10
11 %LaTeX Interpreter if ==1 (requires LaTeX to be installed if compiled on Octave)
12 latex_interpreter=0;
13
14 %-----
15 % CRRA Utility Function (c.^(1-gamma)-1)/(1-gamma)
16 %-----
17 gamma=1.0; %gamma>0
18 if gamma==1
19     u = @(c) log(c);
20 else
21     u = @(c) (c.^(1-gamma)-1)./(1-gamma);
22 end
23
24 %-----
25 % Parameters
26 %-----
27 beta=0.9;      %discount factor
28 A=1;           %Solow Residual
29 alpha=0.30;    %capital elasticity of output
30 delta=0.25;   %depreciation rate
31
32 %-----
33 % Grid on Capital (State Variable: Current Capital Level)
34 %-----
35 k_ss=((alpha*A)/(1/beta-(1-delta)))^(1/(1-alpha)); %steady state capital
36 c_ss=A*k_ss^alpha-delta*k_ss;                    %steady-state consumption
37 k_min=0.25*k_ss;                                %minimum node
38 k_max=1.75*k_ss;                                %maximum node
39 dimK=1000;                                       %number of nodes
40 gridK=linspace(k_min,k_max,dimK);               %uniform grid over capital
41
42 %-----
43 % Consumption and Current Utility Matrices
44 %-----
45 % Consumption can be recovered from current and next period capital.
46 % Creating the (dimK,dimK)-matrix such that an (i,j) element is
47 % the amount of consumption associated with
48 % k=gridK(i) (current state is the i-th node) and
49 % k_prime=gridK(j) (next period's state is the j-th node).
50 % Note that a choice variable is the next-period state.
51 consumption = A*(gridK'*ones(1,dimK)).^alpha + (1-delta)*gridK'*ones(1,dimK)-ones(dimK,1)*gridK;
52 util=zeros(dimK,dimK);
53 for i=1:dimK
54     for j=1:dimK
55         if consumption(i,j)>0
56             util(i,j)=u(consumption(i,j));
57         else
58             util(i,j)=u(consumption(2,1))-10^3;

```

```

59     end
60 end
61 end
62 %alternative
63 %util = ( u(consumption(2,1)) - 10^3 ) .* ones(dimK,dimK);
64 %mask = consumption > 0;
65 %util(mask) = u(consumption(mask));
66
67 %-----
68 % Setting Parametric Values for the Value Function Iteration
69 %-----
70 max_iter=1000; %The Maximum Number of Iterations
71 toler=0.1^10; %Tolerance Value
72 converge=0;
73 % converge is a binary variable, where it takes 1 when the value
74 % function iteration ended within the maximum number of rounds (max_iter).
75 % This is optional.
76
77 %-----
78 % Initialize the Value Function (Initial Guess)
79 %-----
80 % V is a (1, dim_K)-matrix with (1,i)-th element denoting
81 % the value (of a candidate value function)
82 % associated with state being i (precisely, gridK(i)).
83 % Here, simply let V be the vector of zero.
84 V=zeros(1,dimK);
85
86 tic %start stopwatch timer (optional)
87 for i=1:max_iter
88     %Compute the LHS of the Bellman equation newV
89     [newV,index]=max( (util + beta*ones(dimK,1)*V)' );
90     %
91     if norm(newV-V,Inf)<toler %alternative: if max(abs(newV-V))<toler
92         converge=1;
93         break;
94     end %end of if
95     V=newV;
96     %
97 end %end of for i=1:max_iter
98 num_policy=A*gridK.^alpha+(1-delta)*gridK-gridK(index); %Policy function (c=w-w')
99 comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)
100
101 %-----
102 % Checking Whether the Value Function Converged (Optional)
103 %-----
104 if output_file==1
105     dfile="deterministic_Ramsey_growth.txt";
106     if exist(dfile, 'file')
107         delete(dfile);
108     end
109     diary(dfile)
110     diary on
111 end
112
113 if converge==1
114     fprintf('The value function iteration converged with %d iterations.\n', i);
115     fprintf('The value function iteration took %f seconds.\n', comp_time);
116 else

```

```

117     fprintf('The value function iteration did not converge.\n');
118 end
119
120 if output_file==1
121     diary off
122 end
123
124 %-----
125 % Plotting Policy Function
126 %-----
127 figure()
128 if delta==1 && gamma==1
129     exact_policy=(1-alpha*beta)*A*gridK.^alpha;
130     hold on
131     plot(gridK,num_policy,'LineWidth',1);
132     plot(gridK,exact_policy,'--','LineWidth',1);
133     hold off
134     if latex_interpreter==1
135         legend('Numerical','Analytical','Location','southeast','Interpreter','latex');
136     else
137         legend('Numerical','Analytical','Location','southeast');
138     end
139 else
140     plot(gridK,num_policy,'LineWidth',1);
141 end
142 grid on
143 axis([min(gridK) max(gridK) 0.95*min(num_policy) 1.05*max(num_policy)]);
144 if latex_interpreter==1
145     title('Optimal Policy Function','Interpreter','latex','FontSize',14);
146     xlabel('Capital','Interpreter','latex','FontSize',12);
147     ylabel('Consumption','Interpreter','latex','FontSize',12);
148 else
149     title('Optimal Policy Function','FontSize',14);
150     xlabel('Capital','FontSize',12);
151     ylabel('Consumption','FontSize',12);
152 end
153
154 %-----
155 % Plotting Value function
156 %-----
157 figure()
158 if delta==1 && gamma==1
159     exact_V= (1/(1-beta))*((alpha*beta/(1-alpha*beta))*log(alpha*beta) + log(1-alpha*beta)
+log(A)/(1-alpha*beta) )+(alpha/(1-alpha*beta))*log(gridK);
160     hold on
161     plot(gridK,V,'LineWidth',1);
162     plot(gridK,exact_V,'--','LineWidth',1);
163     hold off
164     if latex_interpreter==1
165         legend('Numerical','Analytical','Location','southeast','Interpreter','latex');
166     else
167         legend('Numerical','Analytical','Location','southeast');
168     end
169 else
170     plot(gridK,V,'LineWidth',1);
171 end
172 grid on
173 axis([min(gridK) max(gridK) min(V) max(V)]);

```

```

174 if latex_interpreter==1
175     title('Value Function','Interpreter','latex','FontSize',14);
176     xlabel('Capital','Interpreter','latex','FontSize',12);
177     ylabel('Value','Interpreter','latex','FontSize',12);
178 else
179     title('Value Function','FontSize',14);
180     xlabel('Capital','FontSize',12);
181     ylabel('Value','FontSize',12);
182 end
183
184 %-----
185 % Computing Optimal Consumption/Capital Paths
186 %-----
187 num_sim=2;                %number of simulations
188 num_period=30+1;          %number of periods for each simulation
189 Kindex=zeros(num_sim,num_period); %node index
190 Ksim=zeros(num_sim,num_period);  %value of capital
191 Csim=zeros(num_sim,num_period);  %value of consumption
192 %
193 Ksim(1,1)=0.5*k_ss;    %initial capital level for the first simulation
194 Ksim(2,1)=1.5*k_ss;    %initial capital level for the second simulation
195
196 for i=1:num_sim
197     for t=1:num_period
198         %Find the node closest to a given state
199         [~, Kindex(i,t)]=min(abs(Ksim(i,t)-gridK));
200         %Capital and consumption at time t
201         Ksim(i,t)=gridK(Kindex(i,t));
202         Csim(i,t)=num_policy(Kindex(i,t));
203         if t<num_period
204             Ksim(i,t+1)=A*Ksim(i,t).^alpha+(1-delta)*Ksim(i,t)-Csim(i,t);
205         end
206     end
207 end
208
209 for i=1:num_sim
210     figure()
211     hold on
212     plot(0:num_period-1,Csim(i,:), 'LineWidth',1);
213     plot(0:num_period-1,c_ss*ones(1,num_period), '--', 'LineWidth',1);
214     grid on
215     hold off
216     if latex_interpreter==1
217         xlabel('Period','Interpreter','latex','FontSize',12);
218         ylabel('Consumption','Interpreter','latex','FontSize',12);
219         title('Optimal Consumption Path','Interpreter','latex','FontSize',14);
220         %legend('Optimal Path','Steady State','location','best','Interpreter','latex');
221     else
222         xlabel('Period','FontSize',12);
223         ylabel('Consumption','FontSize',12);
224         title('Optimal Consumption Path','FontSize',14);
225         %legend('Optimal Path','Steady State','location','best');
226     end
227 end
228
229 for i=1:num_sim
230     figure()
231     hold on

```

```

232 plot(0:num_period-1,Ksim(i,:), 'LineWidth',1);
233 plot(0:num_period-1,k_ss*ones(1,num_period), '--', 'LineWidth',1);
234 grid on
235 hold off
236 if latex_interpreter==1
237     xlabel('Period', 'Interpreter', 'latex', 'FontSize',12);
238     ylabel('Capital', 'Interpreter', 'latex', 'FontSize',12);
239     title('Optimal Capital Path', 'Interpreter', 'latex', 'FontSize',14);
240     %legend('Optimal Path','Steady State','location','best','Interpreter','latex');
241 else
242     xlabel('Period', 'FontSize',12);
243     ylabel('Capital', 'FontSize',12);
244     title('Optimal Capital Path', 'FontSize',14);
245     %legend('Optimal Path','Steady State','location','best');
246 end
247 end
248
249 figure()
250 hold on
251 plot(Ksim(1,:),Csim(1,:), '->', 'LineWidth',1);
252 plot(Ksim(2,:),Csim(2,:), '<- ', 'LineWidth',1);
253 plot(gridK,A*gridK.^alpha-delta*gridK, 'LineWidth',1);
254 plot(gridK,A*gridK.^alpha+(1-delta)*gridK-k_ss, 'LineWidth',1);
255 plot(k_ss,c_ss, 'ko', 'LineWidth',1);
256 grid on
257 hold off
258 xlim([k_min k_max]);
259 ylim([0.25*c_ss, 1.75*c_ss]);
260 if latex_interpreter==1
261     xlabel('$k$', 'Interpreter', 'latex', 'FontSize',12);
262     ylabel('$c$', 'Interpreter', 'latex', 'FontSize',12);
263     legend('$k_{0}=0.5 \ k^{\ast}$', '$k_{0}=1.5 \ k^{\ast}$', '$\Delta k_{t+1}=0$', '$\Delta c_{t+1}=0$', '$(k^{\ast}, c^{\ast})$', 'Interpreter', 'latex', 'location', 'northwest');
264     title('Ramsey Growth Model: Phase Diagram', 'Interpreter', 'latex', 'FontSize',14);
265 else
266     xlabel('k', 'FontSize',12);
267     ylabel('c', 'FontSize',12);
268     legend('k_0 =0.5 k*', 'k_0=1.5 k*', '\Delta k_{t+1}=0', '\Delta c_{t+1}=0', '(k*,c*)', 'location', 'northwest');
269     title('Ramsey Growth Model: Phase Diagram', 'FontSize',14);
270 end
271

```